

# Indice

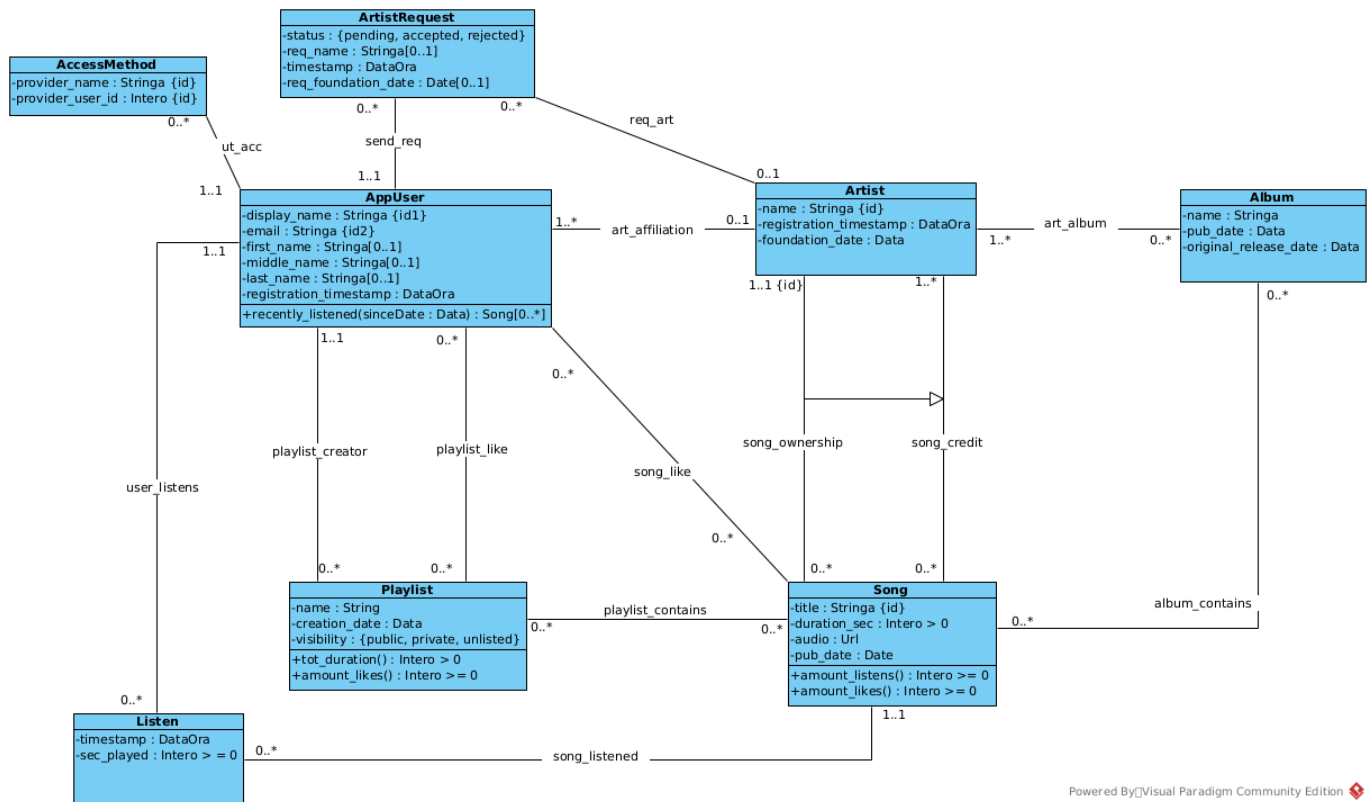
---

- **Analisi Concettuale dei Requisiti**
  - Diagramma delle Classi (Class Diagram)
  - Specifica dei tipi di dato
  - Vincoli Esterni
    - [V.Artist.registrato\_dopo\_fondazione]
    - [V.art\_album.pubblicazione\_dopo\_fondazione]
    - [V.song\_credit.pubblicazione\_dopo\_fondazione]
    - [V.Album.rilascio\_originale\_prima\_di\_pubblicazione]
    - [V.art\_album.artista\_fondato\_prima\_rilascio\_album]
    - [V.art\_album.artista\_registrato\_prima\_pubblicazione\_album]
    - [V.Listen.data\_di\_ascolto\_valida]
    - [V.Listen.non\_ascolta\_piu\_della\_durata]
    - [V.ArtistRequest.tipo\_richiesta\_esclusivo]
    - [V.ArtistRequest.richiesta\_dopo\_registrazione\_ut]
    - [V.ArtistRequest.richiesta\_dopo\_registrazione\_art]
  - Specifica delle Classi
    - Specifica della classe AppUser
    - Specifica della classe Playlist
    - Specifica della classe Song
  - Diagramma degli Use-Case
  - Specifica degli Use-Case
    - Strumenti di Autenticazione e Autorizzazione
    - Strumenti di Utilizzo Playlist e Musiche
    - Strumenti di Interazione
    - Use-Case Strumenti Gestione Catalogo Musicale
    - Use-Case Strumenti Richiesta Artista
    - Use-Case Strumenti di Moderazione
- **Ristrutturazione**
  - Diagramma delle Classi Ristrutturato
  - Specifica dei tipi di dato (Ristrutturata)
  - Vincoli Ristrutturati
    - [V.song\_owner.IS\_A\_song\_credit]
  - Specifica delle Classi
    - Specifica della classe Song
    - Specifica della classe Playlist
    - Implementazione dei Vincoli

## Analisi Concettuale dei Requisiti

---

### Diagramma delle Classi (Class Diagram)



Powered By [Visual Paradigm Community Edition]

## Specifica dei tipi di dato

- Url: Stringa che rappresenta un URL http o https secondo la sintassi RFC 3986

## Vincoli Esterni

[V.Artist.registrato\_dopo\_fondazione] Un Artista può registrarsi solo dopo la sua data di fondazione

Per ogni  $a:Artist$  deve essere vero che  $a.foundation\_date \leq a.registration\_timestamp$

[V.art\_album.pubblicazione\_dopo\_fondazione] Un Artista non può pubblicare un Album se non è stato fondato

Per ogni  $art:Artist$  e  $alb:Album$ , tali che  $(art, alb):art\_album$ , deve essere vero che  $art.foundation\_date < alb.pub\_date$

[V.song\_credit.pubblicazione\_dopo\_fondazione] Un Artista non può pubblicare una Canzone se non è stato fondato

Per ogni  $a:Artist$  e  $s:Song$ , tali che  $(a, s):song\_credit$ , deve essere vero che  $a.foundation\_date \leq s.pub\_date$

[V.Album.rilascio\_originale\_prima\_di\_pubblicazione] Un Album può essere pubblicato sulla piattaforma solo durante o dopo il suo rilascio

Per ogni  $al:Album$ , deve essere vero che  $al.original\_release\_date \leq al.pub\_date$

[V.art\_album.artista\_fondato\_prima\_rilascio\_album] Il rilascio ufficiale di un Album deve avvenire dopo la fondazione dei suoi Artisti

Per ogni *al:Album* e *art:Artist*, tali che *(al, art):art\_album*, deve essere vero che *art.foundation\_date* <= *al.pub\_date*

[V.art\_album.artista\_registrato\_prima\_pubblicazione\_album] La pubblicazione di un Album può avvenire solo da Artisti registrati prima della data di pubblicazione dell'Album stesso

Per ogni *al:Album* e *art:Artist*, tali che *(al, art):art\_album*, deve essere vero che *art.registration\_timestamp* < *al.pub\_date*

[V.Listen.data\_di\_ascolto\_valida] Un Ascolto deve esser fatto dopo la registrazione di un Utente e dopo la pubblicazione di una Canzone

Per ogni *u:AppUser*, *l:Listen* e *s:Song*, tali che *(u, l):user\_listens* e che *(s, l):song\_listened*, devono essere vere entrambe le condizioni:

- *s.pub\_date* <= *l.timestamp*
- *u.registration\_timestamp* <= *l.timestamp*

[V.Listen.non\_ascolta\_piu\_della\_durata] Un Utente non può Ascoltare per una durata superiore alla durata della Canzone stessa

Per ogni *l:Listen* e *s:Song*, tali *(s, l):song\_listened*, deve essere vero che *l.sec\_played* <= *s.duration\_sec*

[V.ArtistRequest.tipo\_richiesta\_esclusivo] Una Richiesta deve indicare il nome per un nuovo Artista oppure riferirsi a un Artista già esistente, ma non entrambe o nessuna delle due.

Per ogni *r:ArtistRequest*, deve essere vera esattamente una e una sola (**xor**) delle due seguenti condizioni:

- Esiste un valore per l'attributo *r.req\_name* e, contemporaneamente, un valore per l'attributo *r.req\_foundation\_date*
- Esiste un *a:Artist*, tale che esista il link *(r, a):req\_art*

[V.ArtistRequest.richiesta\_dopo\_registrazione\_ut] Un Utente non può Richiedere di diventare un Artista prima della sua registrazione

Per ogni *r:ArtistRequest* e *u:AppUser*, tali che esista il link *(r, u):send\_req* deve essere vero che *u.registration\_timestamp* < *r.timestamp*

[V.ArtistRequest.richiesta\_dopo\_registrazione\_art] Una Richiesta non può riguardare un Artista che si è registrato dopo la Richiesta stessa

Per ogni *r:ArtistRequest* e *a:Artist*, tali che esista il link *(r, a):req\_art* deve essere vero che *a.registration\_timestamp* < *r.timestamp*

## Specifica delle Classi

### Specifica della classe AppUser

recently\_listened(sinceDate : Data): Song[0..\*]

Pre: Deve essere vero che *sinceDate* < Oggi

Post: L'operazione non modifica i dati. Il risultato è così definito:

- Sia  $L$  l'insieme di tutti i  $l:Listen$  tali che esiste il link  $(this, l):user\_listens$  e per i quali sia vero che **sinceDate** <  $l.timestamp$
- **result** è l'insieme di tutte le  $s:Song$ , tali che esiste il link  $(l, s):song\_listened$ , per almeno un oggetto  $l$  appartenente all'insieme  $L$

## Specifica della classe Playlist

**tot\_duration():** Intero > 0

Pre: Nessuna

Post: L'operazione non modifica i dati. Il risultato è così definito:

- Sia  $S$  l'insieme di tutte le  $s:Song$ , tali che esiste il link  $(this, s):playlist\_contains$
- **result** è la somma di tutte le  $s.duration\_sec$  in  $S$ , espresso in ore, minuti e secondi

**amount\_likes():** Intero > 0

Pre: Nessuna

Post: L'operazione non modifica i dati. Il risultato è così definito:

- Sia  $UserLikes$  l'insieme di tutti gli  $u:AppUser$ , tali che esiste il link  $(u, this):playlist\_like$
- **result** =  $|UserLikes|$

## Specifica della classe Song

**amount\_listens():** Intero > 0

Pre: Nessuna

Post: L'operazione non modifica i dati. Il risultato è così definito:

- Sia  $L$  l'insieme di tutti i  $l:Listen$ , tali che esista il link  $(l, this):song\_listened$
- **result** =  $|L|$

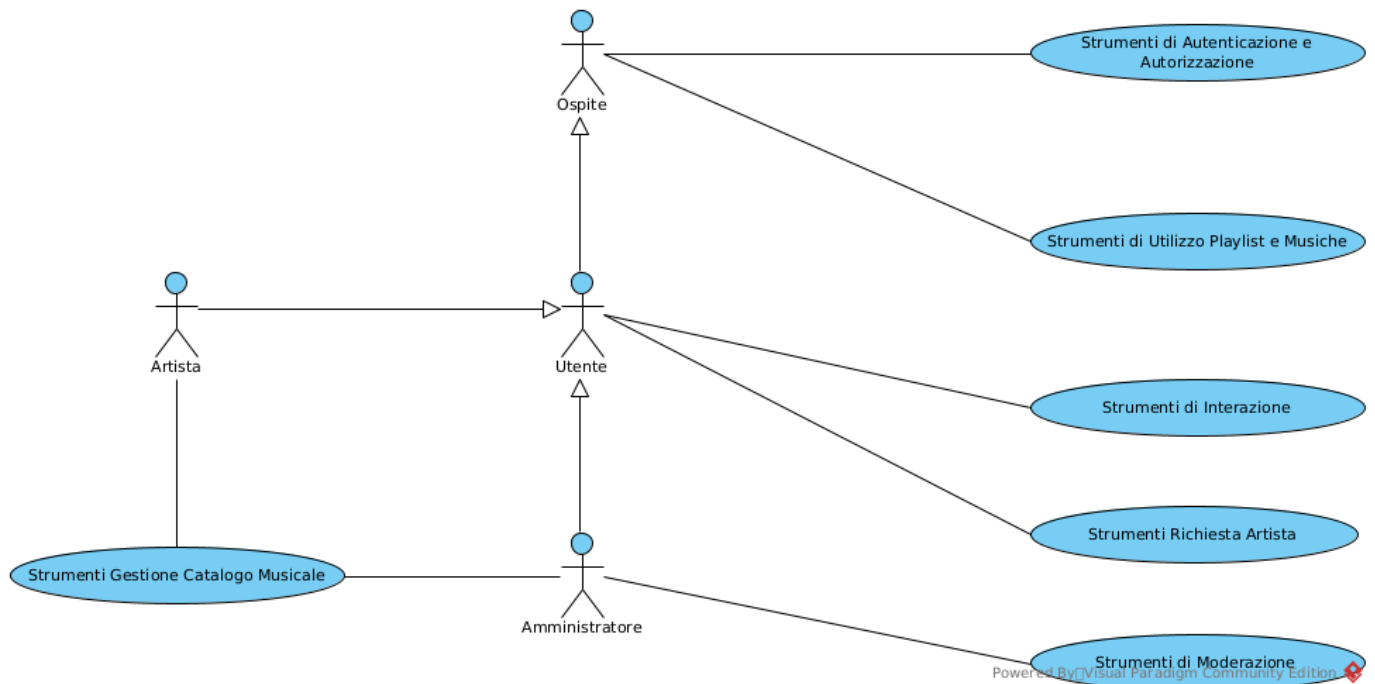
**amount\_likes():** Intero > 0

Pre: Nessuna

Post: L'operazione non modifica i dati. Il risultato è così definito:

- Sia  $UserLikes$  l'insieme di tutti gli  $u:AppUser$ , tali che esiste il link  $(u, this):song\_like$
- **result** =  $|UserLikes|$

## Diagramma degli Use-Case



## Specifica degli Use-Case

### Strumenti di Autenticazione e Autorizzazione

registra(email: Stringa, disp\_name: Stringa, f\_name: Stringa[0..1], m\_name: Stringa[0..1], l\_name: Stringa[0..1], provider\_name : Stringa, provider\_user\_id : Intero) : AppUser

Pre:

- Non deve esistere alcun *u:AppUser* tale che *u.email* = **email**
- Non deve esistere alcun *u:AppUser* tale che *u.display\_name* = **disp\_name**
- **email** e **disp\_name** non devono essere vuoti
- Non deve esistere alcun *am:AccessMethod* tale che *am.provider\_name* = **provider\_name** e *am.provider\_user\_id* = **provider\_user\_id**.

Post:

- Viene creato un nuovo oggetto *am:AccessMethod*, con valori *am.provider\_name* = **provider\_name** e *am.provider\_user\_id* = **provider\_user\_id**
- Viene creato e restituito un nuovo oggetto *res:AppUser*, con valori *res.email* = **email**, *res.display\_name* = **disp\_name**, *res.first\_name* = **f\_name**, *res.middle\_name* = **m\_name**, *res.last\_name* = **l\_name** e *res.registration\_timestamp* = Oggi in questo momento
- Viene creato il link *(am, res):ut\_acc*

associa\_metodo\_accesso(ut: AppUser, provider\_name: Stringa, provider\_user\_id: Intero) : AccessMethod

Pre:

- Non deve esistere alcun *am:AccessMethod* tale che *am.provider\_name* = **provider\_name** e *am.provider\_user\_id* = **provider\_user\_id**.

Post:

- Viene creato e restituito un nuovo oggetto *res:AccessMethod*, con valori *res.provider\_name* = **provider\_name** e *res.provider\_user\_id* = **provider\_user\_id**
- Viene creato il link *(res, ut):ut\_acc*

## Strumenti di Utilizzo Playlist e Musiche

*crea\_playlist(ut: AppUser, name: Stringa, vis: Stringa) : Playlist*

Pre:

- Il valore di **vis** deve appartenere all'insieme {public, private, unlisted}
- **name** non deve essere vuoto

Post:

- Viene creato e restituito un nuovo oggetto *res:Playlist*, con valori *res.name* = **name**, *res.creation\_date* = Oggi e *res.visibility* = **vis**
- Viene creato il link *(ut, res):playlist\_creator*

*aggiungi\_branco\_a\_playlist(ut: AppUser, pl: Playlist, s: Song)*

Pre:

- Deve esistere il link *(ut, pl):playlist\_creator*
- Non deve esistere il link *(pl, s):playlist\_contains*

Post:

- Viene creato il link *(pl, s):playlist\_contains*

## Strumenti di Interazione

*registra\_ascolto(ut: AppUser, s: Song, sec: Intero)*

Pre:

- Deve essere vero che **sec** >= 0 e **sec** <= **s.duration\_sec**

Post:

- Viene creato un nuovo oggetto *l:Listen*, con valori *l.timestamp* = Oggi in questo momento e *l.sec\_played* = **sec**
- Viene creato il link *(ut, l):user\_listens*
- Viene creato il link *(s, l):song\_listened*

*like\_canzone(ut: AppUser, s: Song)*

Pre:

- Non deve esistere il link *(ut, s):song\_like*

Post:

- Viene creato il link *(ut, s):song\_like*

like\_playlist(ut: AppUser, pl: Playlist)

Pre:

- Non deve esistere il link *(ut, pl):playlist\_like*

Post:

- Viene creato il link *(ut, pl):playlist\_like*

## Use-Case Strumenti Gestione Catalogo Musicale

pubblica\_canzone(art\_owner: Artist, art\_credits: Artist[1..\*], song\_title: Stringa, duration: Intero >= 0, audio\_url: Url): Song

Pre:

- **art\_owner** deve essere incluso nell'insieme degli artisti **art\_credits**
- Per ogni *art:Artist* in **art\_credits**, deve essere vero che *art.foundation\_date* <= Oggi
- **song\_title** non deve essere vuoto

Post:

- Viene creato e restituito un nuovo oggetto *res:Song*, con valori *res.title* = **song\_title**, *res.duration\_sec* = **duration**, *res.audio* = **audio\_url** e *res.pub\_date* = Oggi
- Per ogni *art:Artist* in **art\_credits**, vengono creati i link *(art, res):song\_credit*
  - Tra questi, il link *(art\_owner, res)* viene specializzato come istanza anche dell'associazione *song\_ownership*

pubblica\_album(artists: Artist[1..\*], songs: Song[0..\*], alb\_name: Stringa, orig\_rel\_date: Data): Album

Pre:

- **alb\_name** non deve essere vuoto
- **orig\_rel\_date** <= Oggi
- Per ogni *art:Artist* in **artists**, deve essere vero che *art.foundation\_date* <= **orig\_rel\_date**

Post:

- Viene creato e restituito un nuovo oggetto *res:Album*, con valori *res.name* = **alb\_name**, *res.original\_release\_date* = **orig\_rel\_date** e *res.pub\_date* = Oggi
- Per ogni *art:Artist* in **artists**, vengono creati i link *(art, res):art\_album*
- Per ogni *s:Song* in **songs**, vengono creati i link *(res, s):album\_contains*

## Use-Case Strumenti Richiesta Artista

invia\_richiesta\_nuovo\_artista(ut: AppUser, req\_name: Stringa, req\_foundation\_date : Data) : ArtistRequest

Pre:

- Non deve esistere alcun *r:ArtistRequest* tale che *(ut, r):send\_req* e che (*r.status* = 'pending' oppure *r.status* = 'accepted').
- **req\_name** non deve essere vuoto
- Deve essere vero che **req\_foundation\_date** <= Oggi

Post:

- Viene creato e restituito un nuovo oggetto *res:ArtistRequest*, con valori *res.status* = 'pending', *res.req\_name* = **req\_name**, *res.req\_foundation\_date* = **req\_foundation\_date** e *res.timestamp* = Oggi in questo momento
- Viene creato il link *(ut, res):send\_req*

*invia\_richiesta\_artista\_esistente*(*ut*: AppUser, *art*: Artist) : ArtistRequest

Pre:

- Non deve esistere alcun *r:ArtistRequest* tale che *(ut, r):send\_req* e che (*r.status* = 'pending' oppure *r.status* = 'accepted').

Post:

- Viene creato e restituito un nuovo oggetto *res:ArtistRequest*, con valori *res.status* = 'pending', *res.req\_name* = **req\_name** e *res.timestamp* = Oggi in questo momento
- Viene creato il link *(ut, res):send\_req*
- Viene creato il link *(res, art):req\_art*

## Use-Case Strumenti di Moderazione

*valuta\_richiesta*(*req*: ArtistRequest, *accepted*: Booleano)

Pre:

- Deve essere vero che **req.status** = 'pending'

Post:

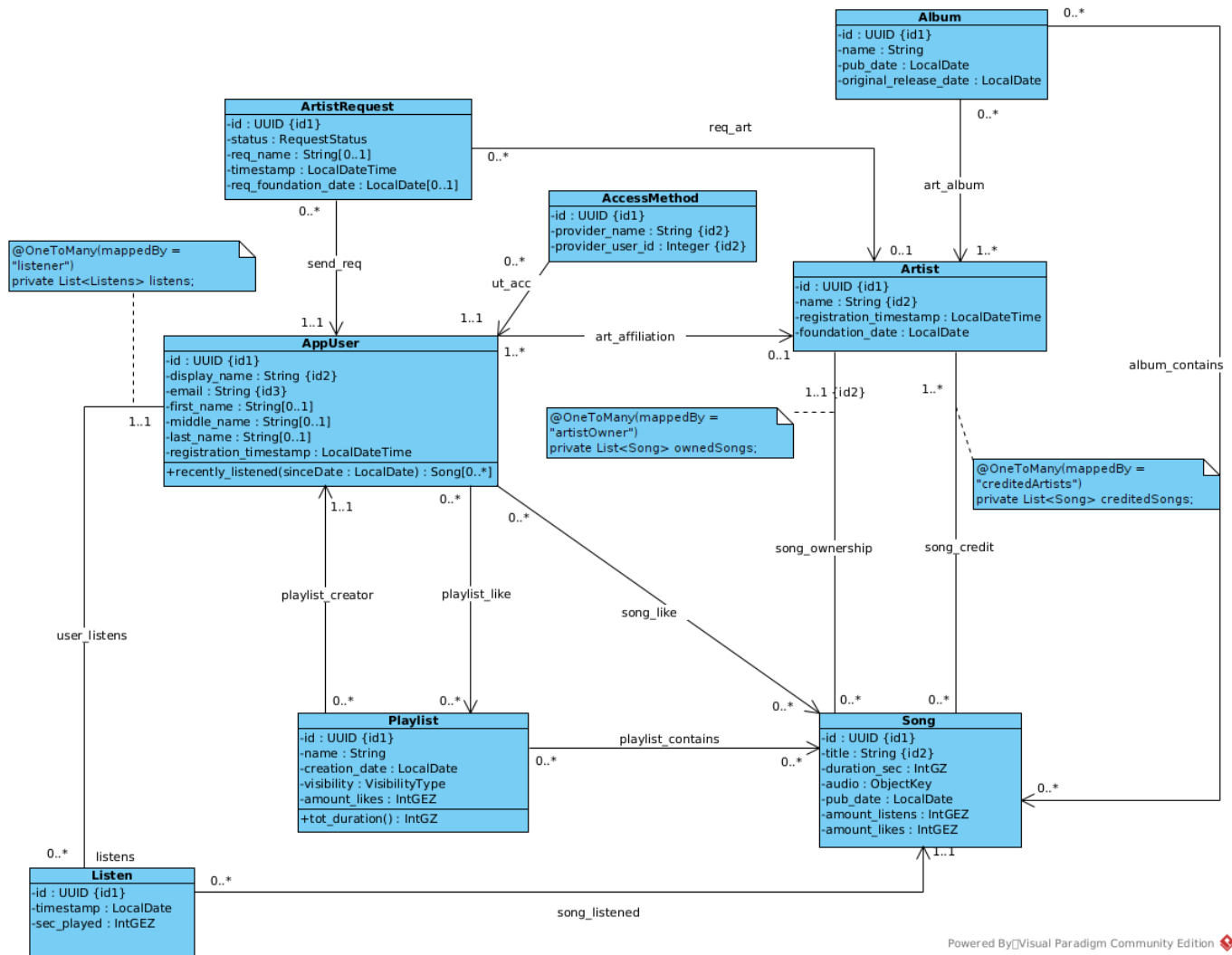
- Se il valore di **accepted** è falso, allora **req.status** diventa uguale a 'rejected'
- Se il valore di **accepted** è vero, allora **req.status** diventa uguale a 'accepted'. Inoltre:
  - Sia *u:AppUser*, tale che esista il link *(u, req):send\_req*
  - Se esiste un *art:Artist*, tale che esista il link *\_(req, art):req\_art*, viene creato il link *(u, art):art\_affiliation*
  - Se non esiste alcun link dell'associazione *req\_art* che coinvolga **req**
    - Viene creato un nuovo oggetto *new\_art:Artist*, con valori *new\_art.name* = **req.req\_name**, *new\_art.foundation\_date* = **req.req\_foundation\_date** e *new\_art.registration\_timestamp* = Oggi in questo momento

## Ristrutturazione

---

### Diagramma delle Classi Ristrutturato





## Specifica dei tipi di dato (Ristrutturata)

- Tipo **IntGZ**

```

@Positive
@Column(nullable = false)
private Integer nomeAttributo;

```

- Tipo **IntGEZ**

```

@PositiveOrZero
@Column(nullable = false)
private Integer nomeAttributo;

```

- Tipo **RequestStatus**

```

public enum RequestStatus {
    PENDING,
    ACCEPTED,
}

```

```
REJECTED
```

```
}
```

- Tipo **VisibilityType**

```
public enum VisibilityType {
    PUBLIC,
    PRIVATE,
    UNLISTED
}
```

- Tipo **ObjectKey**

```
@Pattern(
    regexp = "[a-zA-Z0-9!_.*'()/- ]+$"
)
private String objectKey;
```

## Vincoli Ristrutturati

*NB: Solo i nuovi vincoli o quelli che sono cambiati rispetto all'analisi verranno riportati qui. Se un vincolo non è riportato, la sua logica non è cambiata*

[V.song\_owner.IS\_A\_song\_credit] Un proprietario del brano deve anche essere un artista accreditato

Per ogni *art:Artist* e *s:Song*, tali che *(art, s):song\_ownership*, deve esistere anche il link *(art, s):song\_credit*

## Specifica delle Classi

*NB: Solo le nuove specifiche o quelle che sono cambiati rispetto all'analisi verranno riportati qui. Se una specifica non è riportata, la sua logica non è cambiata*

### Specifica della classe Song

Nel passaggio all'implementazione relazionale, le operazioni di calcolo aggregato sono state sostituite da attributi a ridondanza controllata per ottimizzare le prestazioni in lettura del database:

**Song.amount\_listens** e **Song.amount\_likes**: Sostituiscono le operazioni correlate dell'analisi concettuale. Questi campi verranno aggiornati dinamicamente dall'applicazione ad ogni nuovo inserimento o rimozione delle entità/relazioni **Listen** e **song\_like**.

### Specifica della classe Playlist

Nel passaggio all'implementazione relazionale, le operazioni di calcolo aggregato sono state sostituite da attributi a ridondanza controllata per ottimizzare le prestazioni in lettura del database:

**Playlist.amount\_likes:** Sostituisce l'operazione correlata dell'analisi concettuale. Questo campo verrà aggiornato dinamicamente dall'applicazione ad ogni nuovo inserimento o rimozione della relazione `playlist_like`

## Implementazione dei Vincoli

Nel passaggio al design con Spring JPA e database relazionale, l'integrità del modello sarà garantita su più livelli:

- Vincoli di dominio: saranno controllati nell'applicazione usando le annotazioni di Bean Validation (es. `@Positive`, `@Pattern`) nelle classi `@Entity`.
- Vincoli strutturali: saranno controllati dal database tramite Foreign Key e vincoli UNIQUE.
- Vincoli complessi inter-classe e temporali: saranno controllati con regole più complesse, tramite la logica dei metodi dello strato `@Service` prima di salvare i dati nel database.